

Introducción a Pandas

— Sesión 2 — Fundamentos de Pandas
para Análisis de Datos
Docente: José Zevallos



Objetivos de aprendizaje

Al finalizar la sesión, el estudiante:

- comprende los conceptos fundamentales de Pandas para el análisis de datos;
- utiliza `iloc` y `loc` para indexar y seleccionar datos de un `DataFrame`;
- agrupa datos con `groupby()` y realiza agregaciones;
- usa el método `map()` para aplicar funciones a las columnas;
- fusiona `DataFrames` mediante operaciones de `merge()` con diferentes tipos de uniones (`left`, `right`, `inner`, `outer`).



Ruta de la sesión

01.

Introducción
a Pandas

02.

Indexing con
iloc y loc

03.

Agrupamiento
y agregación

04.

Mapeo de
columnas



01.

□ Introducción a Pandas



¿Qué es Pandas?

Pandas es una librería en Python que proporciona estructuras de datos y herramientas de análisis de datos.

Las estructuras principales son:

- ▶ `DataFrame`: estructura tabular bidimensional;
- ▶ `Series`: una columna de datos.

Pandas permite realizar operaciones como:

- ▶ indexación, filtrado y selección de datos;
 - ▶ agrupamiento y agregación de datos;
 - ▶ fusión y combinación de conjuntos de datos.
-

Creación de DataFrames

Podemos crear un DataFrame manualmente utilizando un diccionario de listas.

Ejemplo: Ventas de Frutas

```
import pandas as pd
fruit_sales = pd.DataFrame(
    data={'Apples': [35, 41],
          'Bananas': [21, 34]},
    index=['2017 Sales', '2018 Sales'])
```

- ▶ **data:** define las columnas y sus valores.
 - ▶ **index:** asigna etiquetas a las filas.
-

Lectura de datos (Fuentes)

Pandas permite cargar datos desde múltiples orígenes:

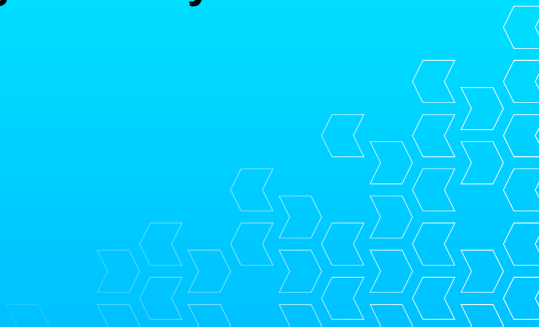
- ▶ **CSV Local:** `pd.read_csv('archivo.csv')`
- ▶ **Desde URL:** `pd.read_csv('https://url.com/data.csv')`
- ▶ **SQL (SQLite):**

```
import sqlite3  
conn = sqlite3.connect("database.db")  
df = pd.read_sql_query("SELECT * FROM tabla", conn)
```

Es fundamental verificar siempre la ruta del archivo o la cadena de conexión.

02.

☰ Indexing con iloc y loc



Uso de `iloc` y `loc`

Pandas permite acceder a los datos de un `DataFrame` mediante índices con `iloc` (indexación posicional) y `loc` (indexación basada en etiquetas).

Ejemplos:

- ▶ `df.iloc[2]`: accede a la fila en la posición 2;
- ▶ `df.loc['A']` accede a la fila con la etiqueta 'A';
- ▶ `df.iloc[:, 1]`: accede a todas las filas en la segunda columna.

Esto permite extraer información específica de manera rápida y eficiente.

Ejemplos de Selección y Filtrado

Aplicando `loc` para selección específica y filtrado condicional:

Selección de fragmentos y columnas:

▶ `df = reviews.loc[0:99, ['country', 'variety']]`

Filtrado Booleano:

▶ `italy = reviews.loc[reviews.country == 'Italy']`

Nota: `loc` incluye el último elemento en el slicing (0:99 incluye la fila 99, a diferencia de las listas de Python).

03.

↑ ≡ Agrupamiento y agregación



Agrupamiento con groupby()

El método `groupby()` permite agrupar datos según una o más columnas y aplicar funciones de agregación.

Ejemplo:

- ▶ `df.groupby('columna').sum()`: agrupa por 'columna' y suma los valores numéricos de otras columnas.
- ▶ `df.groupby(['columna1', 'columna2']).mean()`: agrupa por dos columnas y calcula la media.

Las funciones más comunes son:

- ▶ `sum()`, `mean()`, `count()`, `std()`.
-

Agregación y Ordenamiento

Combinando groupby con funciones de ordenamiento:

Mejor puntuación por precio:

- ▶ `best = reviews.groupby('price')['points'].max().sort_index()`

Conteo por múltiples categorías:

- ▶ `counts = reviews.groupby(['country',
 'variety']).size().sort_values(ascending=False)`

Ordenamiento avanzado:

- ▶ `sorted = extremes.sort_values(by=['min', 'max'], ascending=False)`

04.

Mapeo de columnas



Uso de map() para aplicar funciones

El método `map()` permite aplicar funciones a las columnas de un `DataFrame`.

Ejemplo:

- ▶ `df['columna'].map(lambda x: x * 2)`: multiplica cada valor de la columna por 2.
- ▶ `df['columna'].map(1: 'uno', 2: 'dos')`: mapea valores específicos.

Este método es útil para realizar transformaciones rápidas sobre datos de una columna.

05.

Fusión de DataFrames



Fusionar DataFrames con merge()

El método `merge()` permite combinar dos DataFrames mediante una o más columnas comunes.

Tipos de uniones:

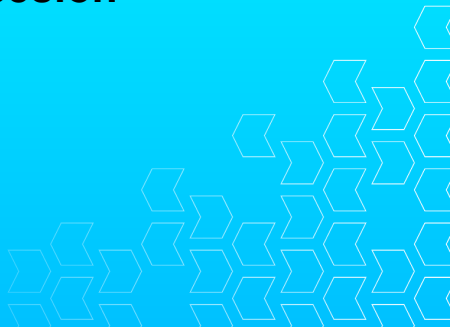
- ▶ `left`: mantiene todos los registros del DataFrame de la izquierda.
- ▶ `right`: mantiene todos los registros del DataFrame de la derecha.
- ▶ `inner`: solo los registros que coinciden en ambos DataFrames.
- ▶ `outer`: mantiene todos los registros de ambos DataFrames.

Ejemplo:

- ▶ `df1.merge(df2, on='columna', how='left')`: realiza una unión tipo left.
-

06.

✓ **Cierre de la sesión**



Ideas de cierre

En esta sesión trabajamos con los siguientes temas clave:

- ▶ `iloc` y `loc` para indexar y seleccionar datos;
- ▶ `groupby()` para agrupar y aplicar agregaciones;
- ▶ `map()` para aplicar funciones a las columnas;
- ▶ `merge()` para fusionar DataFrames.

Con estos conceptos, ya podemos empezar a analizar y manipular datos más complejos utilizando Pandas.
